

Bootstrap/Primordial class Loader :-

It is responsible for loading all the predefined .class files that means all API(Application Programming Interface) level predefined classes are loaded by Bootstrap class loader.

It has the highest priority because Bootstrap class loader is the super class for Platform class loader.

It loads the classes from the following path
C -> Program files -> Java -> JDK -> lib -> jrt-fs.jar

Platform/Extension class loader :

Before java 9V, This class loader was known as Extension class loader because It was loading the .class file from the following path.

C -> Program files -> Java -> JDK -> lib -> ext (extension) -> 3rd Party jar file. [ojdbc14.jar]

It has highest priority than Application class loader.

From Java 9V onwards we have a new concept called JPMS (Java Platform Module System) so, instead of using jar file we can directly use module.
Example : java.base, java.compiler and so on.

Command to create the jar file :
jar cf FileName.jar FileName.class [* .class]

[If we want to compile more than one java source file at a time then the command is : javac *.java]

Application/System class loader :

It is responsible to load all userdefined .class file into JVM memory.

It has the lowest priority because it is the sub class Platform class loader.

It loads the .class file from class path level or environment variable.

Note :-

If all the class loaders are failed to load the .class file into JVM memory then we will get a Runtime exception i.e java.lang.ClassNotFoundException.

```
package com.ravi;

public class Test
{
    public static void main(String[] args)
    {
        IO.println("Test.class file is loaded by :");
        IO.println(Test.class.getClassLoader());

        IO.println(".....");
        IO.println("Super class of Application class loader :");
        IO.println(Test.class.getClassLoader().getParent());
    }
}
```

```
Test.class file is loaded by :
jdk.internal.loader.ClassLoaders$AppClassLoader@29453f44
.....
Super class of Application class loader :
jdk.internal.loader.ClassLoaders$PlatformClassLoader@8efb846
```

How Delegation Hierarchy algorithm works :-

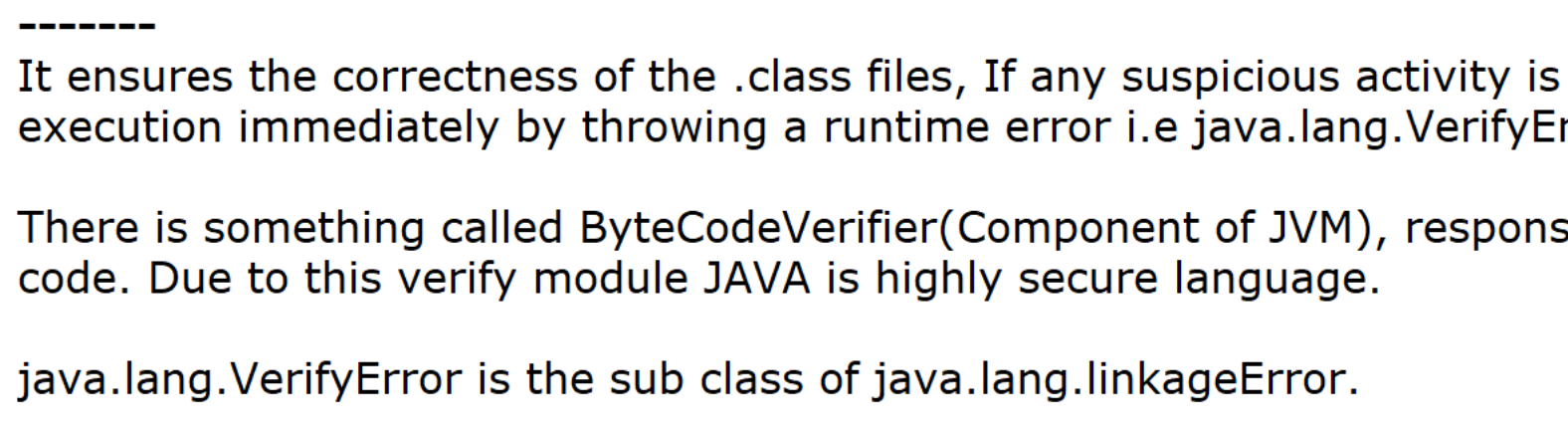
Whenever JVM makes a request to class loader sub system to load the required .class file into JVM memory, first of all, class loader sub system makes a request to Application class loader, Application class loader will delegate(by pass) the request to the Platform class loader, Platform class loader will also delegate the request to Bootstrap class loader.

Bootstrap class loader will load the .class file from lib folder(jrt-fs.jar) and then by pass the request back to Platform class loader, Platform class loader will load the .class file from ext folder(*.jar [3rd party jar file]) OR JPMS (Java Platform Module System) and by pass the request back to Application class loader, It will load the .class file from environment variable into JVM memory.

Note : java.lang.Object is the first class to be loaded into JVM Memory.

The class loader sub system follows

Delegation Hierarchy Algorithm



Linking :

verify

It ensures the correctness of the .class files. If any suspicious activity is there in the .class file then It will stop the execution immediately by throwing a runtime error i.e java.lang.VerifyError.

There is something called ByteCodeVerifier(Component of JVM), responsible to verify the loaded .class file i.e byte code. Due to this verify module JAVA is highly secure language.

java.lang.VerifyError is the sub class of java.lang.linkageError.